

AV1 - CB13 - Programação 1 - 2026.1 - Prof. Marcelo Maia

Questão 1: Validador de Códigos Promocionais

Uma plataforma de vendas online possui regras estritas para a validação de códigos promocionais inseridos pelos usuários. Um código só é considerado válido se atender a **todas** as seguintes regras simultaneamente:

1. Ter exatamente 8 caracteres;
2. Começar com o prefixo "PROMO";
3. Terminar com um número par;
4. Não conter nenhum espaço em branco.

Você deve criar um programa que receba o valor de uma compra e um código, verifique essas regras e aplique um desconto. Se o código for válido, o desconto será de 20% sobre o valor da compra. Caso contrário, o desconto será de 0%.

Formato da Entrada:

1. Uma string representando o código promocional digitado;
2. Um número real representando o valor original da compra.

Formato da Saída:

- Se o código for válido: Código válido! O valor final com desconto é R\$ {valor_final}
- Se o código for inválido: Código inválido! O valor final é R\$ {valor_original}

Exemplos:

Entrada	Saída
PROM0124 100.00	Código válido! O valor final com desconto é R\$ 80.00

PROM0123 50.00	Código inválido! O valor final é R\$ 50.00
PROM0 24 200.00	Código inválido! O valor final é R\$ 200.00
PR012348 150.00	Código inválido! O valor final é R\$ 150.00

Questão 2: Monitoramento de Qualidade do Ar

Um sensor de qualidade do ar emite medições de poluição continuamente. O programa deve ler essas medições até que o sensor seja desligado, evento sinalizado pelo envio de uma medição negativa. Após finalizar as leituras, o programa deve calcular a média da poluição e identificar quantas vezes o nível de poluição ultrapassou o limite considerado crítico (> 50.0).

Formato da Entrada:

1. Uma sequência de números reais, um por linha, representando as medições. A entrada é finalizada quando for inserido um valor negativo.

Formato da Saída:

- Se houver medições válidas, imprimir as seguintes três linhas (média com 2 casas decimais):
Foram registradas {quantidade} medições válidas
Média de poluição: {media}
Níveis críticos (> 50.0) detectados: {contagem} vezes
- Se o primeiro valor digitado já for negativo (nenhuma medição válida), imprimir apenas:
Nenhuma medição registrada

Exemplos:

Entrada	Saída
40.5 55.0 30.0 60.0 -1	Foram registradas 4 medições válidas Média de poluição: 46.38 Níveis críticos (> 50.0) detectados: 2 vezes
20.0 15.5 -5	Foram registradas 2 medições válidas Média de poluição: 17.75 Níveis críticos (> 50.0) detectados: 0 vezes
-10.0	Nenhuma medição registrada

Questão 3: Análise de Mapa de Calor Agrícola

Um drone mapeou a temperatura do solo de uma fazenda, dividida em setores dispostos em uma grade, gerando uma matriz de temperaturas. Um agrônomo precisa saber qual é a temperatura média global da fazenda e quantos setores estão com a temperatura estritamente acima dessa média, indicando possível risco de seca.

Formato da Entrada:

1. Dois números inteiros, **L** e **C**, em linhas separadas, representando respectivamente o número de linhas e colunas da matriz mapeada;
2. Nas **L** linhas seguintes, os valores de temperatura (reais) dos **C** setores daquela linha, separados por espaços em branco.

Formato da Saída:

- A primeira linha com a média global (com 2 casas decimais): **Temperatura média da fazenda: {media}°C**
- A segunda linha com a contagem: **Setores em risco (acima da média): {quantidade}**

Exemplos:

Entrada	Saída
2 3 25.0 26.0 27.0 28.0 29.0 30.0	Temperatura média da fazenda: 27.50°C Setores em risco (acima da média): 3
3 2 35.5 35.5 30.0 30.0 35.5 35.5	Temperatura média da fazenda: 33.67°C Setores em risco (acima da média): 4

Questão 4: Auditoria de Sistemas

Uma empresa sofreu uma falha de segurança e precisa comparar os registros de acesso de dois servidores diferentes (Servidor A e Servidor B). Você deve analisar as listas de usuários (identificados por seus IDs alfanuméricos) que acessaram cada servidor e determinar:

1. Quantos usuários acessaram **ambos** os servidores;
2. Quantos usuários acessaram **apenas** o Servidor A;
3. O total de usuários únicos envolvidos na auditoria de ambos os servidores.

Formato da Entrada:

1. Uma string contendo os IDs dos usuários do Servidor A, separados por vírgula;
2. Uma string contendo os IDs dos usuários do Servidor B, separados por vírgula.

Formato da Saída:

- Linha 1: `Usuários em ambos os servidores: {quantidade}`
- Linha 2: `Usuários apenas no Servidor A: {quantidade}`
- Linha 3: `Total de usuários únicos na auditoria: {quantidade_total}`

Exemplos:

Entrada	Saída
U101,U102,U103,U104 U103,U104,U105	Usuários em ambos os servidores: 2 Usuários apenas no Servidor A: 2 Total de usuários únicos na auditoria: 5
A1,B2,C3 D4,E5	Usuários em ambos os servidores: 0 Usuários apenas no Servidor A: 3 Total de usuários únicos na auditoria: 5

Questão 5: Sistema de Caixa de Lanchonete

Uma lanchonete deseja automatizar o cálculo da conta dos clientes. O programa deve primeiro registrar o cardápio (produtos e preços) na memória. Em seguida, deve processar o pedido de um cliente, calculando o valor total da compra. Se o cliente pedir um item que não está no cardápio, o programa deve emitir um aviso e ignorar aquele item na soma do total.

Formato da Entrada:

1. Um inteiro `N` indicando a quantidade de itens no cardápio;
2. Nas `N` linhas seguintes, uma string no formato `Produto,Preço` (ex: `Hamburguer,25.50`);
3. Um inteiro `P` indicando a quantidade de itens pedidos pelo cliente;
4. Nas `P` linhas seguintes, o nome exato do produto pedido (uma string por linha).

Formato da Saída:

- Para cada produto pedido que não conste no cardápio, imprimir uma linha de alerta: `Aviso: '{produto}' não encontrado no cardápio`
- Ao final, imprimir o custo total do pedido validado: `Total a pagar: R$ {total}`

Exemplos:

Entrada	Saída
3 Hamburguer,25.50 Batata,10.00 Refrigerante,5.00 4 Hamburguer Suco Hamburguer Refrigerante	Aviso: 'Suco' não encontrado no cardápio Total a pagar: R\$ 56.00
2 Coxinha,6.00 Agua,3.50 2 Coxinha Coxinha	Total a pagar: R\$ 12.00

Guia de Referência Rápida:

Categoria	Funções, Métodos e Operadores	Descrição Resumida
Entrada e Saída	<code>print(), input()</code>	Exibe mensagens na tela, lê dados do teclado.
Tipos Básicos	<code>int(), float(), str(), bool()</code>	Funções para conversão de tipos: numéricos, textuais, lógicos.
Aritmética	<code>+, -, *, /, //, %, **</code>	Soma, subtração, multiplicação, divisão, quociente, resto, exponenciação.
Lógica	<code>and, or, not</code>	Operadores lógicos universais: E, OU, NÃO.
Relacional	<code>==, !=, >, <, >=, <=, is, is not</code>	Operadores de comparação de valores.
Coleções (Geral)	<code>len(), min(), max(), sum()</code>	Funções para extrair: tamanho, mínimo, máximo, somatório.
Sequências	<code>in, not in, +, *, [i], [i:p:p]</code>	Verifica pertinência, concatena, replica, acessa por índice, fatia.
Laços	<code>while, for ... in ..., range(i, p, p)</code>	Laço indefinido, laço definido, gerador de sequências numéricas.
	<code>break, continue</code>	Interrompe a execução: de um laço, de uma iteração.
Strings (str)	<code>.upper(), .lower(), .split(sep)</code>	Converte para maiúsculas, converte para minúsculas, divide a string com base num delimitador.
	<code>.find(sub), .rfind(sub), .count(sub)</code>	Busca 1ª ocorrência, busca última ocorrência, conta o n° de ocorrências de uma substring.
	<code>.replace(antigo, novo)</code>	Gera uma cópia da string substituindo as ocorrências de um termo por outro.
Listas (list)	<code>.append(val), .insert(i, val)</code>	Adiciona um item no fim da lista, insere item em uma posição específica i .
	<code>.remove(val), .pop(i)</code>	Remove 1ª ocorrência do valor, remove/retorna o item contido no índice i .
Conjuntos (set)	<code>set(), .add(val), .update(it)</code>	Conjunto vazio, adiciona um item, adiciona múltiplos itens oriundos de outro iterável.
	<code>.remove(val), .discard(val)</code>	Remove item gerando erro caso não exista, remove sem gerar erro caso não exista.
	<code><=, <, >=, >, .isdisjoint()</code>	Verifica se é: subconjunto, subconjunto próprio, superconjunto, superconjunto próprio, conjuntos disjuntos.
Dicionários (dict)	<code>d[chave] = val, .pop(chave)</code>	Cria (ou atualiza) um novo par chave-valor, remove o par apontado pela chave.
	<code>.get(chave, [padrão])</code>	Recupera valor, retornando valor padrão caso a chave não exista.